M.A.I. CONSULTING · AI FLUENCY CURRICULUM

Topics, terms, concepts and commands taught at each level

Companion to the curriculum outlines v0.1 (17 Oct 2026). Levels 1 and 2 are starting points found by the readiness assessment and are not taught; level 3 is the minimum training target. Days are the published cumulative benchmarks. Tool names are examples from the approved list a client adopts; the method is the same on any of them, and commands are shown for the tools named.

Level	Topics (modules)	Terms and concepts	Commands, prompts and actions
Level 1 · Not started Starting point, not taught	Placement only. The readiness assessment records where the person is; no modules.	The seven levels; a task, not a job, as the unit of AI value; the 4Ds as four words (Delegation, Description, Discernment, Diligence).	No commands. The learner arrives with the five tasks they do most often and a baseline time for each (pre-work for level 3).
Level 2 · Occasional use Starting point, not taught	Placement only. Typical habits: a one-line prompt, free tier, no checks, no log.	Why a one-line prompt gives a generic answer; why the free tier is not the organisational plan; "it sounded right" as the usual failure.	No commands. Level 3 replaces these habits; nothing from level 2 is carried forward unchanged.
Level 3 · Structured use 3 days · minimum target	3.1 Placement and task inventory 3.2 What a model is and is not 3.3 Data rules and the approved tool 3.4 Delegation, task by task 3.5 Description: the full brief 3.6 Iteration habits 3.7 The four checks and calibrated trust 3.8 Role workflow, end to end 3.9 Diligence: log, disclose, own 3.10 Clinic on tasks 2 and 3 3.11 Role track (A to E) 3.12 Personal playbook and proof review 3.13 Close	Prediction from patterns; cut-off date; no memory unless added; no files unless given; fluent falsehood; context window; one job per chat; fast versus strong model tier; web search versus own knowledge; the 19-point finding; four data classes (Public, Internal, Confidential, Restricted) and the who-is-harmed test; never-paste list; free tier versus organisational plan; approved tool and its settings; delegate / partially delegate / never; handover point before the first judgement call; the full brief (role, audience, purpose, constraints, format, sources, what good looks like); give the source, never describe it; iteration (options, critique, one change); the four checks (figures, facts, language, defensibility); plausible figure in no source; reading as an editor; which limit did I hit; verification tax; calibrated trust; one-line log; disclosure (name the task and the safeguard); the sender owns it; AI is not a co-author; personal playbook; role tracks (reports, defensible research, meetings, M&E, translation); 90-day re-timing.	Chat assistant on the organisational plan (Copilot, Claude, ChatGPT or Gemini as approved). Start a new chat per job. Choose the model tier. Attach the file rather than pasting a description. Turn web search on or off deliberately. Brief template, seven lines. Follow-ups taught verbatim: "Ask me your questions before you start." "Give me three options, not one." "Critique this without rewriting it." "Change only this paragraph." "Calculate from the table; do not estimate." "Quote the passage you used, with its page." "Where did this figure come from?" "Write not in source where you have no source." Check record (four lines per task). Log line: tool · task · date · disclosure yes/no. Disclosure sentence pattern. Never-paste list pinned at the desk.
Level 4 · Set up +2 days (5 cumulative)	4.1 Context engineering, not prompt engineering 4.2 Projects: one per workstream 4.3 Custom instructions 4.4 Read-only connectors 4.5 Skills: from five manual steps to one Skill 4.6 Build and test the Skill 4.7 Cross-tool equivalents 4.8 Sharing, versioning, set-up hygiene 4.9 Cold-run proof	Context engineering (what the model has in front of it); persistent set-up; stale files; biasing instructions; memory on or off; Project (standing files and instructions for one workstream); file manifest; custom instructions as rules, not wishes; refusal lines; clarifying-question rule; read-only connector; scope (a folder, not a drive); what a connector exposes; Skill (a written method the chat forgets); trigger, inputs, steps, output, checks; the written method as the readiness test; three known inputs before trust; Projects, Notebooks and Gems as equivalents; choose by where the work lives; version date and changelog; set-up audit; set-up owner; cold run by a colleague; rubric and baseline comparison.	Create a Project (Claude) / Notebook (Copilot or NotebookLM) / Gem (Gemini) per workstream; upload template, two best examples, rules, glossary; write Project instructions. Set custom instructions at account level ("never invent a figure; write not in source"). Check and clear memory; turn it off where policy says so. Connect one folder read-only; list what it exposes. Write the Skill design sheet (trigger · inputs · steps · output · checks); build the Skill on the learner's platform; run it on three known inputs; fix what fails. File name with date; changelog line at the top. Set-up audit checklist (no personal data, no stale file, no silent bias). Cold-run record: who · what · rubric scores · time.

Level	Topics (modules)	Terms and concepts	Commands, prompts and actions
Level 5 · Delegating +1.5 days (6.5 cumulative)	5.1 Agents: what they gain and what they cost 5.2 Blast radius and permissions 5.3 The never list, escalation and the near-miss log 5.4 Brief a whole job 5.5 The five-case test set 5.6 Parallel workstreams and supervision 5.7 Review clinic 5.8 Proof review	Agent (plans, acts with tools, checks, repeats); whole job versus task; less visibility, more verification; deep research; long-running task; agents, subagents and skills; "does the task have to decide what happens next?"; blast radius (what it can reach on its worst day); read, write and send separated; access per task, not per agent; browser agents inherit the logged-in session; MCP connection (reads, writes, who approved); indirect prompt injection; the never list (safeguarding referrals, eligibility, payment authorisation, public statements on live matters, employment decisions); escalation path; 24-hour rule; near-miss log (notice, name, narrow); whole-job brief (inputs, outputs, stopping rules, what to ask about, what never to do); write the method before you buy the agent; five questions before delegating; five-case test set; confidence versus competence; human review at the divergence point, not at the end; batch approvals as oversight in name only; supervision log; cost of the run.	Agent mode in the approved assistant (Claude agent or deep research, Copilot agents, Gemini deep research) or a desktop agent the organisation has approved. Whole-job brief, five parts, written before the run. Permissions note: read / write / send · per task · approver. Set tool permissions to read-only by default; grant write only where output is reviewed before it takes effect; send only with recorded approval. Stop command and stopping rules stated in the brief. Five-case test set: case · expected result · actual result · pass/fail. Run two jobs in parallel on sample material; keep a supervision log. Near-miss entry: notice · name · narrow. Escalation message within 24 hours: what · which data · which tool · what has been done.
Level 6 · Building for others +1.5 days (8 cumulative)	6.1 Designing for someone else 6.2 Evaluation 6.3 Connectors and MCP at organisation level 6.4 Injection and exfiltration in depth 6.5 Models, vendors and cost 6.6 Review of the three set-ups 6.7 The operating guide and the review cycle 6.8 Proof review	Design sheet (inputs, method, outputs, refusal cases, operating guide, named owner, test set); agent, skill or tool and what each costs to build, run and govern; apprenticeship problem (Draft, Review, Own); rubric; spot check; judge the decisions, not just the output; corrections fed back into the set-up; attendance and training record as evidence; permissions record; write access and its approval; logging; trust ladder (sandbox to standard, fallback and owner at each step); document store as attack surface; curated sources; exfiltration; safeguarding language check as a built rule; cost per completed task (including failed attempts, review and rework); reading a vendor benchmark; data terms; exit plan (method, data, workflow, account control); hours to capacity, outcomes and budget lines; operating guide; six-month review and four triggers; rationale ledger; institutional memory.	Design sheet, one page per set-up, with a named owner who is not the builder. Department review rubric (0 to 3 per check). Permissions record: server or connector · what it exposes · read/write · approver · log location. Trust-ladder entry per new feature: sandbox · test on own documents · policy check · listing. Threat walk: "what could an edited document make this set-up do?" Injection note per set-up. Completed-task ledger: task · attempts · review time · rework · cost. Operating guide sections: purpose · inputs · steps · checks · refusal cases · owner · review date · triggers. Rationale ledger: one line per boundary. One-week-away test and new-colleague cold start.
Level 7 · At the terminal +2 days (10 cumulative)	7.1 The terminal and the working folder 7.2 Claude Code without code 7.3 Dry run and batch work 7.4 git in three commands; secrets out of files 7.5 One review across a year of reports 7.6 The repeatable pipeline 7.7 What is still engineering 7.8 Proof: the second person	Terminal (a text window: what to do and where); working folder, one per job; never run on the only copy; CLAUDE.md (what the files are, the rules, the output format, what never to touch); permission modes; "allow all" never on real files; hooks that block deletes and writes outside the folder; slash commands and Skills for repeated jobs; MCP servers in the terminal; dry run ("list what you would change, change nothing"); batch work (rename, restructure, classify, review a hundred files); the run log; commit before and after; what versioning protects; secrets (passwords, tokens, keys) out of files and out of CLAUDE.md; consolidated review with file and page references; spot check against sources; pipeline; runbook; scheduled run and the person who reads the log; failure note; the engineering boundary (database, public interface, other people's credentials, silent-failure integrations).	Shell: cd into the job folder; mkdir a working folder; copy the files in (copy or cp); dir or ls to see what is there. Claude Code: claude to start; /init to create CLAUDE.md, then edit it in plain language; /permissions and a permission mode that asks before every write; /hooks to add a rule that blocks deletion and writes outside the folder; /mcp or claude mcp add for an approved server; a slash command or Skill for the repeated job. Prompts taught verbatim: "List what you would change. Change nothing." then "Now do it and write a log of every change." git: git init once; git add . and git commit -m "before run" before; git status, git diff and git commit -m "after run" after. Secrets scan: search the folder for password, token, key and .env before any commit. Runbook, scheduled run (Task Scheduler or cron) and a deliberate failure test. Codex CLI and Gemini CLI follow the same steps.

Prompts in quotation marks are taught as written and then adapted to the learner's own task. Nothing in this table is a productivity promise; outcomes are measured per engagement by the re-timing at 90 days.